

스터디 소개

본 스터디는 UC Berkeley CS285 Deep Reinforcement Learning 강의를 기반으로, 심층강화학습의 핵심 개념과 주요 알고리즘을 함께 학습하는 것을 목표로 합니다.

현재까지 전체 23개 강의 중 10강까지 진행하며, 모방학습에서 시작해 강화학습의 기본 문제 설정, Policy Gradient, Actor-Critic, Value Function, Q-Learning, 최적제어와 계획까지의 흐름을 정리하였습니다.

매주 세션에서는 강의 내용을 단순히 따라가는데 그치지 않고, 알고리즘이 어떤 문제의식에서 등장했는지와 서로 어떻게 연결되는지를 중심으로 토론하고 있습니다.

주제 소개

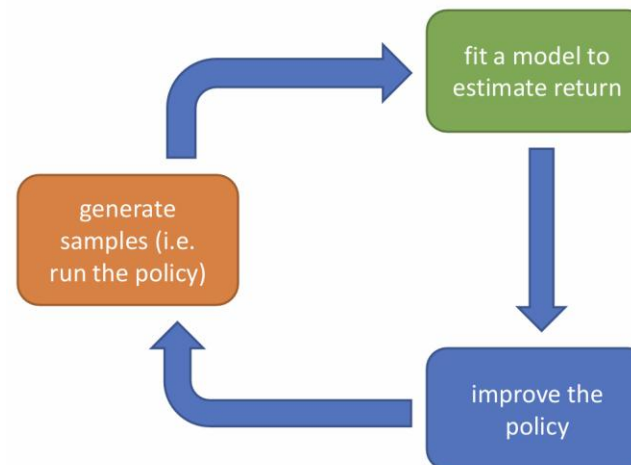
심층강화학습은 에이전트가 환경과 상호작용하며 시행착오를 통해 더 나은 의사결정 전략을 학습하는 알고리즘을 연구하는 분야입니다. 딥러닝이 복잡한 상태와 행동을 표현하는 능력을 제공한다면, 강화학습은 시간에 따라 누적되는 보상을 고려해 행동을 선택하는 문제 설정을 가정합니다.

CS285 Deep RL은 이러한 두 축을 바탕으로, 에이전트가 데이터를 수집하고, 정책과 가치함수를 학습하며, 환경의 구조를 활용해 더 효율적으로 행동하는 방법을 폭넓게 다룹니다. 강의는 먼저 강화학습의 기본 문제 설정과 모방학습을 통해 순차적 의사결정 문제를 소개한 뒤, 정책을 직접 최적화하는 Policy Gradient, 가치함수를 활용하는 Value-Based Method, 두 관점을 결합한 Actor-Critic 방법으로 이어집니다. 이후에는 모델 기반 강화학습, 탐험, 오프라인 강화학습, 제어와 추론의 연결, 역강화학습, 시퀀스 모델, 메타러닝 등으로 주제를 확장하며, 심층강화학습이 로보틱스, 게임, 제어, 생성 모델 등 다양한 문제와 어떻게 연결되는지를 살펴봅니다.

따라서 본 강의는 특정 알고리즘을 개별적으로 익히는 데 그치지 않고, 에이전트가 무엇을 학습하는가, 어떤 데이터로부터 학습하는가, 환경에 대한 지식을 어떻게 활용하는가라는 질문을 중심으로 심층강화학습의 전체 지도를 그리는 과정이라고 볼 수 있습니다. 이번 스터디는 그중 현재까지의 학습 내용(10강)을 바탕으로, 앞으로 다룰 심화 주제들과의 연결점을 함께 정리해 나가고 있습니다.

Policy Gradient

[정책 개선의 학습 루프]



1. Generate samples: 정책 π_θ 를 실제 환경에서 실행하여 여러 trajectory를 수집한다.
2. Estimate return: 수집한 샘플로부터 각 행동 이후에 얻게 될 누적 보상(return)을 추정한다.
3. Improve policy: 높은 return을 만든 행동의 확률은 높이고, 낮은 return을 만든 행동의 확률은 낮추는 방향으로 정책을 업데이트한다.

[Policy Gradient]

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

[REINFORCE algorithm]

REINFORCE algorithm:

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)$ (run it on the robot)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i | \mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

REINFORCE는 가장 기본적인 Policy Gradient 알고리즘이다. 환경에서 얻은 trajectory의 return을 이용해 정책을 직접 업데이트하며, 보상이 큰 행동의 확률을 증가시키는 방식으로 학습한다.

Actor-Critic

[Actor vs. Critic]

- Actor: 현재 상태에서 어떤 행동을 선택할지 결정하는 정책 모델
- Critic: Actor가 선택한 행동이 얼마나 좋은지 평가하는 가치함수 모델
 1. Value function $V^\pi(\mathbf{s})$: 상태 $\mathbf{s}$ 에서 앞으로 얻을 것으로 기대되는 누적 보상
 2. Q function $V^\pi(\mathbf{s}, \mathbf{a})$: 상태 $\mathbf{s}$ 에서 행동 $\mathbf{a}$ 를 선택했을 때 기대되는 누적 보상
 3. Advantage function $A^\pi(\mathbf{s}, \mathbf{a})$: 특정 행동이 현재 상태의 평균적인 기대보다 얼마나 더 좋은지 측정

[Actor-Critic algorithm]

batch actor-critic algorithm:

1. sample $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_\theta(\mathbf{a} | \mathbf{s})$ (run it on the robot)
2. fit $\hat{V}_\phi^\pi(\mathbf{s})$ to sampled reward sums
3. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i | \mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Q-Learning

[Q-iteration algorithm]

full fitted Q-iteration algorithm:

1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy
2. set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\phi \leftarrow \arg \min_\phi \frac{1}{2} \sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$

[Q-Learning]

online Q iteration algorithm:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$